Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 1: Implementation of Source Code Management Using Git Commands Through Github

Submitted By:

Ujjwal Rana

Roll no. 43

Submitted To:

Er. Rajendra Bahadur Thapa

Lab Report

Title: Source Code Management Using Git and GitHub

Objective

To understand and implement Source Code Management (SCM) using Git and GitHub. The objectives of this lab are to:

- Configure Git with user credentials.
- Create and initialize a local Git repository.
- Track project files using Git.
- Commit changes to the local repository.
- Connect the local repository to a remote GitHub repository.
- Push the project to GitHub for remote version control.

Tools Required

- Git
- GitHub Account
- Git Bash or Terminal
- Visual Studio Code (Optional)

Theory

Source Code Management (SCM) is the process of tracking, organizing, and managing changes made to source code during software development. It enables developers to maintain different versions of a project, collaborate efficiently, and restore previous versions when necessary.

Git is a distributed version control system that allows developers to record changes locally and maintain a complete history of a project. It supports branching, merging, and efficient collaboration among team members.

GitHub is a cloud-based hosting platform for Git repositories. It provides remote storage, collaboration features, version history, issue tracking, and project management tools, making it easier for developers to work together on software projects.

Procedure

Step 1: Configure Git

Configure Git by setting the global username and email address that will be associated with all commits.

Step 2: Create and Initialize a Repository

Create a new project directory and initialize it as a Git repository using the git init command.

Step 3: Create Project Files

Create a sample project file (README.md) containing basic content.

Step 4: Stage Files

Add the project files to the staging area using the git add command.

Step 5: Commit Changes

Save the staged changes to the local repository by creating a commit with a descriptive commit message.

Step 6: Create a GitHub Repository

Create a new repository on GitHub to serve as the remote repository for the local project.

Step 7: Connect Local and Remote Repositories

Add the GitHub repository as the remote origin using the repository URL.

Step 8: Push the Repository

Rename the default branch to main and push the local commits to the GitHub repository.

Step 9: Verify Repository Status

Use Git commands to check the current repository status and review the commit history.

Git Commands (Coding Section)

Configuring github auth

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Create and initialize a repository

```
mkdir MyProject
cd MyProject
git init
```

Create a sample file

```
echo "Hello GitHub!" > README.md
```

Stage files

```
git add .
```

Commit changes

git commit -m "Initial commit"

Connect local repository to GitHub

git remote add origin https://github.com/username/MyProject.git

Rename branch to main

git branch -M main

Push repository to GitHub

git push -u origin main

Check repository status

git status

View commit history

git log

Output

The following tasks were successfully completed:

- Git was installed and configured with the user's credentials.
- A local Git repository was created and initialized.
- Project files were added to the staging area.
- Changes were committed to the local repository.
- A remote GitHub repository was created.
- The local repository was successfully connected to the remote repository.
- The project was pushed to GitHub successfully.
- Repository status and commit history were verified using Git commands.

Result

The experiment successfully demonstrated the use of Git and GitHub for Source Code Management. The local repository was initialized, project files were tracked and committed, and the repository was synchronized with a remote GitHub repository.

Conclusion

This lab provided practical experience with the complete Git workflow used in modern software development. The experiment covered configuring Git, initializing repositories, staging files, creating commits, linking a local repository to GitHub, and pushing code to a remote repository. It demonstrated how Git and GitHub facilitate version control, maintain project history, and support efficient collaboration in Agile software development.